

Web Security Testing

1. ASSESSMENT OVERVIEW

- Target: OWASP Juice Shop
- Target Type: Intentionally Vulnerable Web Application
- Environment: Localhost
- URL: http://localhost:3000
- Assessment Type: Basic Web Application Security Assessment
- Testing Focus: Reconnaissance, HTTP analysis, authentication, authorization, input validation, XSS, file upload validation, and error handling.

2. OBJECTIVE

The objective of this assessment was to practice a basic web application security testing workflow:

1. Application reconnaissance
2. Identification of functionality and input points
3. HTTP request and response analysis
4. Authentication and session testing
5. Authorization testing
6. Input validation testing
7. Cross-Site Scripting testing
8. File upload validation testing
9. Error handling analysis
10. Documentation of security findings

Testing was performed against a locally hosted OWASP Juice Shop instance.

3. TOOLS USED

- Google Chrome
- Chrome Developer Tools (Network, Elements, Console)
- OWASP Juice Shop
- Manual HTTP request and response analysis

4. APPLICATION RECONNAISSANCE

Observed functionality included:

- User registration
- User login/logout
- User profile
- Search functionality
- Product browsing

- Shopping basket
- Product quantity controls
- Password recovery
- Profile image upload
- API endpoints
- User authentication using a session token/cookie

Examples of observed endpoints:

- GET /profile
- POST /profile
- GET /rest/user/whoami
- GET /rest/basket/{id}
- GET /rest/user/security-question?email={email}
- POST /rest/user/reset-password
- POST /profile/image/file

5. AUTHENTICATION TESTING

Invalid login credentials were tested. The application returned a generic message indicating that the email or password was incorrect rather than identifying which specific credential was invalid.

Result: No basic account-enumeration finding was identified during login testing.

6. AUTHORIZATION / ACCESS CONTROL TESTING

Shopping basket endpoints were tested using two separate local accounts.

Account A used Basket ID 6.

Account B used Basket ID 7.

Controlled access tests:

- Account A → Basket 6: 200 OK
- Account A → Basket 7: 401 Unauthorized
- Account B → Basket 7: 200 OK
- Account B → Basket 6: 401 Unauthorized

Result: No cross-account basket access was demonstrated.

7. INPUT VALIDATION TESTING

Search functionality was tested with special characters including: ' " test' % *

Some special-character inputs caused several products to appear in search results. This was documented as an observation, but was not classified as SQL Injection because sufficient evidence of database query manipulation was not established.

8. FINDING F-01 — DOM-BASED CROSS-SITE SCRIPTING (XSS)

- Severity: High
- Category: Cross-Site Scripting (XSS)
- Status: Confirmed
- Description: The search functionality was found to process attacker-controlled input from the URL fragment in a way that allows JavaScript execution in the browser.
- Payload: `<img src=x onerror=alert(1)>`
- Resulting URL:
`http://localhost:3000/#/search?q=%3Cimg%20src%3Dx%20onerror%3Dalert(1)%3E`

The JavaScript alert executed successfully and continued to execute when the URL was refreshed or reopened.

Steps to Reproduce:

1. Open the local OWASP Juice Shop instance.
2. Navigate to search.
3. Enter: `<img src=x onerror=alert(1)>`
4. Submit the search.
5. Observe the JavaScript alert.
6. Refresh or reopen the resulting URL.
7. Observe that the JavaScript executes again.

Impact:

An attacker may be able to execute JavaScript in a victim's browser if a malicious URL can be delivered to the victim. Depending on the application context, XSS can enable manipulation of page content, phishing interfaces, actions using the victim's browser session, and other client-side attacks.

Recommendation:

- Treat user-controlled data as untrusted.
- Safely encode untrusted output.
- Avoid unsafe DOM APIs for untrusted data.
- Sanitize HTML where HTML input is genuinely required.
- Apply an appropriate Content Security Policy as defense in depth.
- Review the client-side search rendering logic responsible for processing the URL fragment.

9. FINDING F-02 — SENSITIVE INFORMATION DISCLOSURE

- Severity: High
- Category: Sensitive Data Exposure / Information Disclosure
- Status: Confirmed
- Description: During the password-reset process, `POST /rest/user/reset-password` returned a complete user object in the HTTP response.

The response included sensitive/internal fields such as:

- id
- email
- role
- password (password hash)
- totpSecret
- lastLoginIp
- isActive
- createdAt
- updatedAt

The password field contained a hash rather than plaintext, but password hashes are still sensitive credential material and should not be returned to the client.

Observed request:

POST /rest/user/reset-password

Impact:

Exposed password hashes may potentially be subjected to offline password-cracking attacks. Returning additional internal fields also increases unnecessary information exposure.

Recommendation:

The reset-password endpoint should return only the minimum information required by the client. Sensitive fields such as password and totpSecret should never be included in normal API responses. API responses should use an explicit allowlist of safe fields rather than serializing complete database user models.

10. FINDING F-03 — IMPROPER ERROR HANDLING / INFORMATION DISCLOSURE

- Severity: Low
- Category: Information Disclosure / Improper Error Handling
- Status: Observed
- Description: Invalid operations generated detailed server-side error pages. An invalid profile image upload produced “500 Error: Illegal file type” and exposed internal application information including a server-side source path and route filename. A similar error was observed on profile functionality.

Information exposed included:

- Express framework information
- Internal source file paths
- Application route filenames
- Server-side stack trace information

Impact:

Detailed errors can reveal application architecture, frameworks, internal file structure, source-code locations, and backend implementation details, potentially assisting further attacks.

Recommendation:

- Return generic error messages to clients.
- Log detailed errors server-side.
- Disable detailed stack traces in production.
- Use centralized error handling.
- Avoid exposing internal filesystem paths and source-code locations.

11. FILE UPLOAD VALIDATION

Profile image upload functionality was tested with:

- JPG — Accepted
- PNG — Accepted
- TXT — Rejected
- TXT renamed to JPG — Rejected

A text file renamed with a .jpg extension was rejected.

Result: No file-type bypass was demonstrated. The application did not rely solely on the filename extension during this test.

The rejection did, however, generate a detailed server error, which contributed to the error-handling finding.

12. BUSINESS LOGIC TESTING

Shopping basket quantity controls were tested.

Observed limits:

Minimum: 1

Maximum: 5

The interface did not allow quantities below 1 or above 5 during the tested interaction.

Result: No business-logic vulnerability was demonstrated through the tested interface.

13. SECURITY CONTROLS / POSITIVE OBSERVATIONS

- Invalid login attempts returned a generic authentication error.
- Cross-account basket access was rejected with 401 Unauthorized.
- A text file renamed to .jpg was rejected.
- Basket quantities were restricted to the observed range of 1–5.

14. FINDINGS SUMMARY

- F-01 — DOM-Based Cross-Site Scripting — High — Confirmed
- F-02 — Sensitive Information Disclosure — High — Confirmed
- F-03 — Improper Error Handling / Information Disclosure — Low — Observed

Tests with no demonstrated vulnerability:

- Basket IDOR / cross-account access
- Basic login enumeration
- File-upload extension bypass
- Basket quantity manipulation

Inconclusive:

- Search special-character behavior / possible injection

15. CONCLUSION

The assessment demonstrated a basic web application security testing workflow using OWASP Juice Shop as a controlled and intentionally vulnerable target.

Two significant vulnerabilities were confirmed:

1. DOM-Based Cross-Site Scripting through search functionality.
2. Sensitive information disclosure through the password-reset response, including a password hash and other internal user fields.

An additional information disclosure issue was observed through detailed server-side error responses.

A key lesson was that unusual application behavior should not automatically be classified as a vulnerability. Potential issues should be tested, validated, supported with evidence, and documented accurately.

Future testing areas include API authorization, HTTP request manipulation, session security, CSRF, SQL injection, SSRF, security headers, and Burp Suite-based testing.

Findings Summary Table

ID	Finding	Severity	Status
F-01	DOM-Based Cross-Site Scripting (XSS)	High	Confirmed
F-02	Sensitive Information Disclosure	High	Confirmed
F-03	Improper Error Handling / Information Disclosure	Low	Observed